November 2, 2026 at 19:15

1. Introduction. Exercise 7.2.2.1–147 takes the thirty coloured cubes of exercise 146 and asks what solids they can build. A "brick" of size $l \times m \times n$ is an assembly of $l \cdot m \cdot n$ of the cubes in which touching faces agree in colour and each of the six outer sides is a single solid colour. Part (a) asks for a brick of size $2 \times 3 \times 5$ that uses all thirty; part (b) asks for a catalogue of every brick there is.

The answer supplies that catalogue: for each shape and each pattern of face colours it lists how many essentially different bricks have 1, 2, 4 or 8 automorphisms. Twenty-four such lines are printed. This program computes them all, together with the smaller claims the answer makes along the way.

Everything here follows the answer's own recipe. Positions and faces are named by the even/odd coordinates of exercise 145; the exact cover problem is the one answer 147(b) describes; and the count is divided by 720 in the way the answer prescribes, by leaving only a single option for the first cell.

```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strings"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Declarations 5 ⟩
⟨ Functions 7 ⟩
func main() {
    ⟨ Read the command line 2 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. Five modes. The census checks the piece set on its own, before anything is built out of it. The printed mode reads back the two arrays that answer 147(a) displays. The catalogue mode is the main event. The near mode looks at the two shapes the answer mentions in its closing parenthesis, and the six mode counts the bricks whose outer sides show six different colours—a statistic that turns out to be worth having.

⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "catalog",
    "census, printed, catalog, near, six, solve, or brute")
size := flag.String("b", "2x2x4", "brick size lxmxn")
nopin := flag.Bool("nopin", false, "keep all 720 options for the first cell")
dump := flag.Bool("dump", false, "list every essentially different brick")
flag.Parse()
noPin, dumpAll = *nopin, *dump
var l, m, n int
fmt.Sscanf(*size, "%dx%dx%d", &l, &m, &n)
```

This code is used in section 1.

3. ⟨ Do what the mode asks 3 ⟩ ≡

```
switch *mode {
case "census":
    census()
case "printed":
    printedCheck()
case "near":
    nearMiss()
case "six":
    distinctSides(brick{l, m, n})
case "solve":
    report(brick{l, m, n})
case "brute":
    bruteSides(l, m, n)
case "catalog":
    ⟨ Run every shape that could possibly work 4 ⟩
default:
    fmt.Println("unknown mode")
}
```

This code is used in section 1.

4. A brick can hold at most thirty cubes, so $lmn \le 30$; nothing else needs to be assumed. Shapes with no solutions print one line and pass by, which is how the absence of $3 \times 3 \times 3$—and of $2 \times 2 \times 6$ and $2 \times 2 \times 7$, which the answer's catalogue also passes over in silence—gets checked.

⟨ Run every shape that could possibly work 4 ⟩ ≡

```
for i := 1; i ≤ 30; i++ {
    for j := i; j ≤ 30; j++ {
        for k := j; k ≤ 30; k++ {
            if i * j * k ≤ 30 {
                report(brick{i, j, k})
            }
        }
    }
}
```

This code is used in section 3.

5. ⟨ Declarations 5 ⟩ ≡

```
var noPin, dumpAll bool
```

This code is also defined in sections 6, 12, 14, 17, 18, 26, 29, 48, 54, 58, 62.

This code is used in section 1.

6. The thirty cubes. Answer 146(b) counts the cubes by pairing up the colours: for each way of splitting $\{a, b, c, d, e, f\}$ into three opposite pairs there are two cubes, mirror images of each other, and $15 \times 2 = 30$. It also names them. The cube called $uu'vv'ww'$ is the one that can be placed with u on top, u' on the bottom, v in front, v' in the back, w at the left and w' at the right.

Reading the constraints the answer gives, the name is built like this: a is always on top, since it is the smallest colour; the colour opposite it may be any of the other five; of the four that remain the smallest goes in front and its partner in the back; and the last pair goes left and right, either way round. That last choice is exactly the choice of mirror image, and $5 \times 3 \times 2 = 30$.

I keep a cube as the colour on each of its six faces, with the faces indexed by direction.

⟨ Declarations 5 ⟩ +≡

```
const (
    dTop = iota
    dBot
    dFront
    dBack
    dLeft
    dRight
)
var dvec = [6][3]int{{0, 0, 1}, {0, 0, −1}, {0, −1, 0}, {0, 1, 0}, {−1, 0, 0}, {1, 0, 0}}
type cube struct {
    name string
    col [6]byte
}
```

7. ⟨ Functions 7 ⟩ ≡

```
func cubes( ) []cube {
    var out []cube
    ⟨ Take each colour in turn as the one opposite a 8 ⟩
    return out
}
```

This code is also defined in sections 9, 10, 13, 15, 16, 19, 20, 21, 22, 24, 30, 32, 34, 35, 36, 37, 39, 40, 44, 46, 51, 53, 56, 57, 61, 64.

This code is used in section 1.

8. The four colours left after a and its opposite are chosen have their smallest member in front; the partner of that one may be any of the other three; and the two that then remain go left and right in either order.

⟨ Take each colour in turn as the one opposite a 8 ⟩ ≡

```
for up := byte('b'); up ≤ 'f'; up++ {
    four := others('a', up, 0, 0)
    for _, vp := range four[1:] {
        last := others('a', up, four[0], vp)
        for _, flip := range []bool { false, true }{
            w, wp := last[0], last[1]
            if flip {
                w, wp = wp, w
            }
            c := cube{col: [6]byte{'a', up, four[0], vp, w, wp}}
            c.name = string(c.col[:])
            out = append(out, c)
        }
    }
}
```

This code is used in section 7.

9. ⟨ Functions 7 ⟩ +≡

```
func others(p, q, r, s byte) []byte {
    var out []byte
    for c := byte('a'); c ≤ 'f'; c++ {
        if c ≠ p ∧ c ≠ q ∧ c ≠ r ∧ c ≠ s {
            out = append(out, c)
        }
    }
    return out
}
```

10. Turning a cube over. The twenty-four rotations of a cube are the signed permutations of the three axes with determinant +1. I store each one as a permutation of the six directions: entry d says where the face that pointed in direction d ends up.

⟨ Functions 7 ⟩ +≡

```
func rotations( ) [ ][6]int {
    var out [ ][6]int
    for _, p := range axisPerms {
        for s := 0; s < 8; s++ {
            ⟨ Assemble one signed permutation and keep the proper ones 11 ⟩
        }
    }
    return out
}
```

11. The sign of the permutation times the product of the reversals is the determinant, and a reflection is no use here: a cube cannot be turned into its mirror image by any motion.

⟨ Assemble one signed permutation and keep the proper ones 11 ⟩ ≡

```
det := permSign(p)
var q [6]int
for d := 0; d < 6; d++ {
    var w [3]int
    for a := 0; a < 3; a++ {
        w[a] = dvec[d][p[a]]
        if s & (1 << a) ≠ 0 {
            w[a] = −w[a]
        }
    }
    q[d] = dirOf(w)
}
for a := 0; a < 3; a++ {
    if s & (1 << a) ≠ 0 {
        det = −det
    }
}
if det > 0 {
    out = append(out, q)
}
```

This code is used in section 10.

12. ⟨ Declarations 5 ⟩ +≡

```
var axisPerms = [ ][3]int{{0, 1, 2}, {0, 2, 1}, {1, 0, 2}, {1, 2, 0}, {2, 0, 1}, {2, 1, 0}}
```

13. ⟨ Functions 7 ⟩ +≡

```
func permSign(p [3]int) int {
    s := 1
    for i := 0; i < 3; i++ {
        for j := i + 1; j < 3; j++ {
            if p[i] > p[j] {
                s = -s
            }
        }
    }
    return s
}
func dirOf(v [3]int) int {
    for i, w := range dvec {
        if w ≡ v {
            return i
        }
    }
    panic("not an axis direction")
}
```

14. Every colouring is a cube. Thirty cubes with twenty-four orientations each make 720 ways of painting six distinct colours on six labelled faces—and there are exactly 6! = 720 such paintings altogether. So the list below should hit every one of them once, and checking that is a good way to be sure the naming rule was read correctly. If a cube had been missed, or one counted twice, the collision would show up here rather than in some later number that nobody could interpret.

⟨ Declarations 5 ⟩ +≡

```
type placement struct {
    name string
    col [6]byte
}
```

15. ⟨ Functions 7 ⟩ +≡

```
func placements() []placement {
    var out []placement
    for _, c := range cubes() {
        for _, r := range rotations() {
            var col [6]byte
            for d := 0; d < 6; d++ {
                col[r[d]] = c.col[d]
            }
            out = append(out, placement{c.name, col})
        }
    }
    return out
}
func byColour() map[[6]byte]string {
    out := map[[6]byte]string{}
    for _, p := range placements() {
        out[p.col] = p.name
    }
    return out
}
```

16. The census also looks up the ten cube names that the two answers print, six of them in answer 146(b) for the row of cubes in (∗) and four more in answer 147(a).

⟨ Functions 7 ⟩ +≡

```
func census() {
    cs, pl, bc := cubes(), placements(), byColour()
    fmt.Printf("%d␣cubes,␣%d␣placements,␣%d␣distinct␣face␣colourings\n",
        len(cs), len(pl), len(bc))
    named := map[string]bool{}
    for _, c := range cs {
        named[c.name] = true
    }
    missing := 0
    for _, w := range printedNames {
        if ¬named[w] {
            fmt.Printf("␣␣the␣answers␣name␣%s,␣which␣I␣do␣not␣have\n", w)
            missing++
        }
    }
    fmt.Printf("all␣%d␣names␣printed␣in␣the␣answers␣are␣in␣the␣set:␣%v\n",
        len(printedNames), missing ≡ 0)
}
```

17. ⟨ Declarations 5 ⟩ +≡

```
var printedNames = []string{"aebfcd", "acbfde", "acbdef", "afbdec", "abcedf",
    "aebcfd", "abcefd", "abcdfe", "abcdef", "acbefd"}
```

18. Coordinates for a brick. Exercise 145 gives the convention that answer 147(a) calls ideal, and it is: let the triple (x, y, z) with $0 \le x \le 2l$, $0 \le y \le 2m$, $0 \le z \le 2n$ stand for a vertex, edge, face or cell of the brick according as it has none, one, two or three odd coordinates. So the cell (i, j, k) sits at $(2i+1, 2j+1, 2k+1)$ and its six faces are one step away along each axis.

A brick may have cells missing, which is needed only at the very end, for the two shapes that come close to a $3 \times 3 \times 3$; everywhere else the set of absent cells is empty.

⟨ Declarations 5 ⟩ +≡

```
type pt [3]int
type brick struct { l, m, n int }
var skip = map[[3]int]bool{}
```

19. ⟨ Functions 7 ⟩ +≡

```
func (b brick) dims() [3]int { return [3]int{b.l, b.m, b.n} }
func (b brick) eachCell(f func(i, j, k int)) {
    for i := 0; i < b.l; i++ {
        for j := 0; j < b.m; j++ {
            for k := 0; k < b.n; k++ {
                if ¬skip[[3]int{i, j, k}] {
                    f(i, j, k)
                }
            }
        }
    }
}
func (b brick) ncells() int {
    n := 0
    b.eachCell(func(i, j, k int) { n++ })
    return n
}
```

20. The faces of a cell, in the same order as the directions.

⟨ Functions 7 ⟩ +≡

```
func (b brick) faces(i, j, k int) [6]pt {
    return [6]pt{
        {2*i+1, 2*j+1, 2*k+2},
        {2*i+1, 2*j+1, 2*k},
        {2*i+1, 2*j, 2*k+1},
        {2*i+1, 2*j+2, 2*k+1},
        {2*i, 2*j+1, 2*k+1},
        {2*i+2, 2*j+1, 2*k+1},
    }
}
func fname(p pt) string { return fmt.Sprintf("F%d.%d.%d", p[0], p[1], p[2]) }
```

21. Listing the face triples in a fixed order gives a canonical way to write a brick down as a string, which is what the tests for sameness will need.

⟨ Functions 7 ⟩ +≡

```
func (b brick) coords() []pt {
    d := b.dims()
    var out []pt
    for x := 0; x ≤ 2 * d[0]; x++ {
        for y := 0; y ≤ 2 * d[1]; y++ {
            for z := 0; z ≤ 2 * d[2]; z++ {
                if x % 2 + y % 2 + z % 2 ≡ 2 {
                    out = append(out, pt{x, y, z})
                }
            }
        }
    }
    return out
}
```

22. The outer sides. A face is on the outside when its cell is on the boundary and the face points away from the brick.

⟨ Functions 7 ⟩ +≡

```
func (b brick) exterior() [6][]pt {
    var out [6][]pt
    b.eachCell(func(i, j, k int) {
        ff := b.faces(i, j, k)
        ⟨ Collect the faces of this cell that face outwards 23 ⟩
    })
    return out
}
```

23. ⟨ Collect the faces of this cell that face outwards 23 ⟩ ≡

```
if k ≡ b.n − 1 {
    out[dTop] = append(out[dTop], ff[dTop])
}
if k ≡ 0 {
    out[dBot] = append(out[dBot], ff[dBot])
}
if j ≡ 0 {
    out[dFront] = append(out[dFront], ff[dFront])
}
if j ≡ b.m − 1 {
    out[dBack] = append(out[dBack], ff[dBack])
}
if i ≡ 0 {
    out[dLeft] = append(out[dLeft], ff[dLeft])
}
if i ≡ b.l − 1 {
    out[dRight] = append(out[dRight], ff[dRight])
}
```

This code is used in section 22.

24. The exact cover problem. Answer 147(b) lays the problem out. There is a primary item for every cell, a secondary item for every one of the thirty cube names, and a secondary item for every face; an option puts one cube, in one orientation, into one cell, and colours that cell's six faces. Two cells that share a face therefore have to agree about its colour, which is the whole point of the colour mechanism.

Six more primary items take care of the outer sides. Each has six options, one per colour, and the option paints every face on that side the same. The answer writes one of them out in full: `top 101:c 103:c 105:c 107:c 109:c 301:c 303:c 305:c 307:c 309:c`.

```
⟨ Functions 7 ⟩ +≡
  func (b brick) xcc()(string, int) {
      var prim, sec []string
      ⟨ Name the items 25 ⟩
      var sb strings.Builder
      sb.WriteString(strings.Join(prim, "␣") + "␣|␣" + strings.Join(sec, "␣") + "\n")
      nopt := 0
      ⟨ Write an option for every cube in every cell 27 ⟩
      ⟨ Write the options that paint the outer sides 28 ⟩
      return sb.String(), nopt
  }
```

25.

```
⟨ Name the items 25 ⟩ ≡
  b.eachCell(func(i, j, k int) {
      prim = append(prim, fmt.Sprintf("P%d.%d.%d", i, j, k))
  })
  prim = append(prim, sideName[:]...)
  for _, c := range cubes() {
      sec = append(sec, "N" + c.name)
  }
  seen := map[string]bool{}
  var fs []string
  b.eachCell(func(i, j, k int) {
      for _, f := range b.faces(i, j, k) {
          if ¬seen[fname(f)] {
              seen[fname(f)] = true
              fs = append(fs, fname(f))
          }
      }
  })
  sort.Strings(fs)
  sec = append(sec, fs...)
```

This code is used in section 24.

26.

```
⟨ Declarations 5 ⟩ +≡
  var sideName = [6]string{"XU", "XD", "XF", "XB", "XL", "XR"}
```

27. Here is where the factor of 720 comes off. Every permutation of the six colours carries a brick to a brick, and the permutation is pinned down completely by what it does to one cube's six faces. So each orbit under those 720 permutations contains exactly one brick whose first cell holds a chosen placement, and keeping only that option counts each orbit once. The answer says as much; the program checks it, by solving one case both ways.

⟨ Write an option for every cube in every cell 27 ⟩ ≡

```
pl := placements()
first := true
b.eachCell(func(i, j, k int) {
    ff := b.faces(i, j, k)
    pinned := first ∧ ¬noPin
    first = false
    for _, p := range pl {
        if pinned ∧ p.col ≠ [6]byte { 'a', 'b', 'c', 'd', 'e', 'f' }{
            continue
        }
        fmt.Fprintf(&sb, "P%d.%d.%d␣N%s", i, j, k, p.name)
        for d := 0; d < 6; d++ {
            fmt.Fprintf(&sb, "␣%s:%c", fname(ff[d]), p.col[d])
        }
        sb.WriteByte('\n')
        nopt++
    }
})
```

This code is used in section 24.

28. ⟨ Write the options that paint the outer sides 28 ⟩ ≡

```
ext := b.exterior()
for d := 0; d < 6; d++ {
    for c := byte('a'); c ≤ 'f'; c++ {
        sb.WriteString(sideName[d])
        for _, f := range ext[d] {
            fmt.Fprintf(&sb, "␣%s:%c", fname(f), c)
        }
        sb.WriteByte('\n')
        nopt++
    }
}
```

This code is used in section 24.

29. Reading a solution back. A solution is easier to think about as the array of answer 147(a): a colour on every face of the brick, interior and exterior alike. The cube in a cell is then whatever the six faces around it say it is, so the array carries all the information there is.

```
⟨ Declarations 5 ⟩ +≡
    type tensor struct {
        b brick
        col map[pt]byte
    }
```

30.

```
⟨ Functions 7 ⟩ +≡
    func (b brick) solve() []tensor {
        in, _ := b.xcc()
        xc := cells.NewXCC()
        var out []tensor
        for sol := range xc.Dance(strings.NewReader(in)).Solutions {
            t := tensor{b, map[pt]byte{}}
            for _, o := range sol {
                ⟨ Copy this option's face colours into the array 31 ⟩
            }
            out = append(out, t)
        }
        return out
    }
```

31.

```
⟨ Copy this option's face colours into the array 31 ⟩ ≡
    for _, it := range o {
        i := strings.IndexByte(it, ':')
        if i < 0 ∨ it[0] ≠ 'F' {
            continue
        }
        var p pt
        fmt.Sscanf(it[1:i], "%d.%d.%d", &p[0], &p[1], &p[2])
        t.col[p] = it[i + 1]
    }
```

This code is used in section 30.

32. Telling bricks apart. Two bricks count as the same when one becomes the other under a rotation or reflection of the box together with a permutation of the colours. The geometric part is easy to list: an axis may be sent to any axis of the same length, and any axis may be reversed.

⟨ Functions 7 ⟩ +≡

```
func (b brick) syms() []func(pt) pt {
    d := b.dims()
    var out []func(pt) pt
    for _, s := range axisPerms {
        if d[0] ≠ d[s[0]] ∨ d[1] ≠ d[s[1]] ∨ d[2] ≠ d[s[2]] {
            continue
        }
        for f := 0; f < 8; f++ {
            ⟨ Add the symmetry that permutes by s and reverses by f 33 ⟩
        }
    }
    return out
}
```

33. ⟨ Add the symmetry that permutes by s and reverses by f 33 ⟩ ≡

```
sg, fl := s, f
out = append(out, func(p pt) pt {
    var q pt
    for a := 0; a < 3; a++ {
        v := p[sg[a]]
        if fl & (1 << a) ≠ 0 {
            v = 2 * d[a] - v
        }
        q[a] = v
    }
    return q
})
```

This code is used in section 32.

34. The colour part is easier still. Renaming the colours in order of first appearance produces a string that two bricks share exactly when a permutation of colours turns one into the other; no search is needed.

⟨ Functions 7 ⟩ +≡

```
func relabel(b brick, col map[pt]byte) string {
    seen := map[byte]byte{}
    var sb strings.Builder
    for _, p := range b.coords() {
        c := col[p]
        if _, ok := seen[c]; ¬ok {
            seen[c] = byte('a' + len(seen))
        }
        sb.WriteByte(seen[c])
    }
    return sb.String()
}
```

35. So the canonical form of a brick is the least of those strings over all the symmetries, and its automorphisms are the symmetries that leave the string alone. For each symmetry there is at most one colour permutation that could work—the image of a single cube decides it—so the automorphism group is a subgroup of the geometric one, and a class of bricks accounts for exactly G/Aut of the solutions the search finds.

⟨ Functions 7 ⟩ +≡

```
func (t tensor) canon()(string, int) {
    me := relabel(t.b, t.col)
    best, naut := "", 0
    for _, g := range t.b.syms() {
        img := map[pt]byte{}
        for p, c := range t.col {
            img[g(p)] = c
        }
        s := relabel(t.b, img)
        if best ≡ "" ∨ s < best {
            best = s
        }
        if s ≡ me {
            naut++
        }
    }
    return best, naut
}
```

36. The face colours. The answer sorts its catalogue by the pattern of colours on the six outer sides, written as three pairs: `aa x bb x cc` when the sides come in three matched pairs, `ab x cd x ef` when all six differ, and so on. The pairs go in the order of the axes; when two axes have the same length they may be swapped, so I take whichever order reads smaller once the colours are renamed.

⟨ Functions 7 ⟩ +≡

```
func (t tensor) sidePairs( ) [3][2]byte {
    d := t.b.dims( )
    var pr [3][2]byte
    for a := 0; a < 3; a++ {
        pr[a][0] = t.faceOn(a, 0)
        pr[a][1] = t.faceOn(a, 2 * d[a])
        if pr[a][0] > pr[a][1] {
            pr[a][0], pr[a][1] = pr[a][1], pr[a][0]
        }
    }
    return pr
}
func (t tensor) faceOn(a, lim int) byte {
    for _, p := range t.b.coords( ) {
        if p[a] ≡ lim {
            return t.col[p]
        }
    }
    return 0
}
```

37. ⟨ Functions 7 ⟩ +≡

```
func (t tensor) sides( ) string {
    d, pr := t.b.dims( ), t.sidePairs( )
    best := ""
    for _, s := range axisPerms {
        if d[0] ≠ d[s[0]] ∨ d[1] ≠ d[s[1]] ∨ d[2] ≠ d[s[2]] {
            continue
        }
        ⟨ Spell the three pairs out in the order s 38 ⟩
    }
    return best
}
```

38. ⟨ Spell the three pairs out in the order s 38 ⟩ ≡

```
seen := map[byte]byte{}
var sb strings.Builder
for a := 0; a < 3; a++ {
    if a > 0 {
        sb.WriteString("␣x␣")
    }
    for _, c := range pr[s[a]] {
        if _, ok := seen[c]; !ok {
            seen[c] = byte('a' + len(seen))
        }
        sb.WriteByte(seen[c])
    }
}
if best == "" ∨ sb.String() < best {
    best = sb.String()
}
```

This code is used in section 37.

39. The answer notices something about these patterns: a colour that shows up twice on the surface always does so on two parallel sides, never on two that meet. That is a claim about every solution of every case, and it costs nothing to test it while the solutions go past.

⟨ Functions 7 ⟩ +≡

```
func (t tensor) oppositeOnly() bool {
    pr := t.sidePairs()
    cnt := map[byte]int{}
    for a := 0; a < 3; a++ {
        cnt[pr[a][0]]++
        cnt[pr[a][1]]++
    }
    for c, k := range cnt {
        if k == 1 {
            continue
        }
        if k > 2 ∨ ¬somePair(pr, c) {
            return false
        }
    }
    return true
}
func somePair(pr [3][2]byte, c byte) bool {
    for a := 0; a < 3; a++ {
        if pr[a][0] == c ∧ pr[a][1] == c {
            return true
        }
    }
    return false
}
```

40. One line of the catalogue. Now everything can be put together. For one shape: solve, group the solutions into classes, and count the classes by pattern and by number of automorphisms, which is precisely the form the answer's summary takes.

```
⟨ Functions 7 ⟩ +≡
    func report(b brick) {
        _, nopt := b.xcc()
        ts := b.solve()
        fmt.Printf("%dx%dx%d:␣%d␣cells,␣%d␣options,␣%d␣solutions\n",
            b.l, b.m, b.n, b.ncells(), nopt, len(ts))
        if len(ts) ≡ 0 {
            return
        }
        ⟨ Check the remark about parallel sides 41 ⟩
        ⟨ Sort the solutions into classes and print the line 42 ⟩
    }
```

41. ⟨ Check the remark about parallel sides 41 ⟩ ≡

```
    bad := 0
    for _, t := range ts {
        if ¬t.oppositeOnly() {
            bad++
        }
    }
    if bad > 0 {
        fmt.Printf("␣␣␣␣%d␣solutions␣repeat␣a␣colour␣on␣sides␣that␣meet\n", bad)
    } else {
        fmt.Printf("␣␣␣␣every␣repeated␣side␣colour␣is␣on␣parallel␣sides\n")
    }
```

This code is used in section 40.

42. ⟨ Sort the solutions into classes and print the line 42 ⟩ ≡

```
type key struct {
    sides string
    aut int
}
class := map[string]tensor{}
for _, t := range ts {
    c, _ := t.canon()
    class[c] = t
}
tally := map[key]int{}
accounted := 0
for _, t := range class {
    _, na := t.canon()
    tally[key{t.sides(), na}]++
    accounted += len(b.syms())/na
}
fmt.Printf("␣␣␣␣|G|␣=␣%d,␣%d␣essentially␣different,␣%d␣solutions␣accounted␣for\n",
    len(b.syms()), len(class), accounted)
⟨ Print one row per pattern of face colours 43 ⟩
```

This code is used in section 40.

43. The answer's table has columns for 1, 2, 4 and 8 automorphisms. Anything outside that range—and the $2 \times 2 \times 2$ case has a brick with twenty-four—gets a note of its own.

⟨ Print one row per pattern of face colours 43 ⟩ ≡

```
var pats []string
for k := range tally {
    if ¬contains(pats, k.sides) {
        pats = append(pats, k.sides)
    }
}
sort.Strings(pats)
for _, s := range pats {
    fmt.Printf("␣␣␣␣%-14s␣(%d,␣%d,␣%d,␣%d)", s,
        tally[key{s, 1}], tally[key{s, 2}], tally[key{s, 4}], tally[key{s, 8}])
    for k, v := range tally {
        if k.sides ≡ s ∧ k.aut ≠ 1 ∧ k.aut ≠ 2 ∧ k.aut ≠ 4 ∧ k.aut ≠ 8 {
            fmt.Printf("␣␣[and␣%d␣with␣%d␣automorphisms]", v, k.aut)
        }
    }
    fmt.Println()
}
if dumpAll {
    ⟨ List every class with its six side colours 45 ⟩
}
```

This code is used in section 42.

44. ⟨ Functions 7 ⟩ +≡

```
func contains(ss []string, s string) bool {
    for _, t := range ss {
        if t ≡ s {
            return true
        }
    }
    return false
}
```

45. When two counts disagree it helps to see the bricks themselves. This lists each class with the colour on each of its six sides and how many colours that makes—a plain fact about a brick that owes nothing to any convention for ordering the pairs.

⟨ List every class with its six side colours 45 ⟩ ≡

```
var keys []string
for k := range class {
    keys = append(keys, k)
}
sort.Strings(keys)
for _, k := range keys {
    t := class[k]
    _, na := t.canon()
    pr := t.sidePairs()
    set := map[byte]bool{}
    for a := 0; a < 3; a++ {
        set[pr[a][0]] = true
        set[pr[a][1]] = true
    }
    fmt.Printf("␣␣␣␣␣␣aut␣%2d␣␣%-14s␣␣%c%c␣%c%c␣%c%c␣␣%d␣colours␣on␣the␣surface\n",
        na, t.sides(), pr[0][0], pr[0][1], pr[1][0], pr[1][1],
        pr[2][0], pr[2][1], len(set))
}
```

This code is used in section 43.

46. Six different colours. The catalogue's disagreements, if there are any, are easier to talk about in a form that involves no canonical ordering at all: of the solutions a shape has, how many show six different colours on the outside? A third model settles that independently of how the classes are sorted. Give each colour a secondary item of its own and hang it on the options that paint a side; a secondary item can be taken at most once, so no two sides can claim the same colour, and with six sides and six colours they must all differ.

⟨ Functions 7 ⟩ +≡

```
func distinctSides(b brick) {
    in, _ := b.xcc()
    lines := strings.Split(strings.TrimRight(in, "\n"), "\n")
    ⟨ Add a colour item to the header and to every side option 47 ⟩
    xc := cells.NewXCC()
    n := 0
    for range xc.Dance(strings.NewReader(strings.Join(lines, "\n") + "\n")).Solutions {
        n++
    }
    fmt.Printf("%dx%dx%d:␣%d␣bricks␣with␣six␣different␣colours␣outside\n",
        b.l, b.m, b.n, n)
}
```

47. ⟨ Add a colour item to the header and to every side option 47 ⟩ ≡

```
head := strings.SplitN(lines[0], "␣|␣", 2)
var used []string
for c := byte('a'); c ≤ 'f'; c++ {
    used = append(used, fmt.Sprintf("U%c", c))
}
lines[0] = head[0] + "␣|␣" + strings.Join(used, "␣") + "␣" + head[1]
for i, ln := range lines[1:] {
    for d := 0; d < 6; d++ {
        if strings.HasPrefix(ln, sideName[d] + "␣") {
            lines[i + 1] = ln + fmt.Sprintf("␣U%c", ln[len(ln) − 1])
        }
    }
}
```

This code is used in section 46.

48. The arrays the answer prints. Answer 147(a) displays two of these arrays: the $3 \times 5 \times 5$ one for the $1 \times 2 \times 2$ brick of the exercise, and the $5 \times 7 \times 11$ one for the magnificent $2 \times 3 \times 5$. Both are worth reading back in, because a transcription that survives the test below is almost certainly right: a single wrong letter would leave some cell holding a colouring that is no cube at all.

```
⟨ Declarations 5 ⟩ +≡
    var printed122 = []string{
        ⟨ The array of the 1 × 2 × 2 brick 49 ⟩
    }
    var printed235 = []string{
        ⟨ The array of the magnificent brick 50 ⟩
    }
```

49.

```
⟨ The array of the 1 × 2 × 2 brick 49 ⟩ ≡
    ".....\n.a.a.\n.....\n.a.a.\n.....",
    ".d.d.\nc.e.c\n.f.f.\nc.d.c\n.e.e.",
    ".....\n.b.b.\n.....\n.b.b.\n.....",
```

This code is used in section 48.

50.

```
⟨ The array of the magnificent brick 50 ⟩ ≡
    ".........\n.a.a.a.a.a.\n.........\n.a.a.a.a.a.\n" +
        ".........\n.a.a.a.a.a.\n.........",
    ".c.c.c.c.c.\nb.f.d.f.d.b\n.e.e.b.b.f.\nb.c.d.f.e.b\n" +
        ".f.f.e.d.d.\nb.e.d.b.e.b\n.c.c.c.c.c.",
    ".........\n.d.b.e.e.e.\n.........\n.d.b.c.c.c.\n" +
        ".........\n.d.b.f.f.f.\n.........",
    ".c.c.c.c.c.\nb.f.d.b.f.b\n.e.e.f.d.d.\nb.c.d.e.f.b\n" +
        ".f.f.b.b.e.\nb.e.d.e.d.b\n.c.c.c.c.c.",
    ".........\n.a.a.a.a.a.\n.........\n.a.a.a.a.a.\n" +
        ".........\n.a.a.a.a.a.\n.........",
```

This code is used in section 48.

51. Each block is one value of x, each line of a block one value of y, and each character one value of z; a dot stands wherever the triple is not a face. Reading it in checks the shape as it goes.

```
⟨ Functions 7 ⟩ +≡
    func readPrinted(b brick, blocks []string)(tensor, error) {
        t := tensor{b, map[pt]byte{}}
        d := b.dims()
        if len(blocks) ≠ 2 * d[0] + 1 {
            return t, fmt.Errorf("want␣%d␣blocks,␣got␣%d", 2 * d[0] + 1, len(blocks))
        }
        for x, blk := range blocks {
            ⟨ Read one block of the array 52 ⟩
        }
        return t, Λ
    }
```

52. ⟨ Read one block of the array 52 ⟩ ≡

```
rows := strings.Split(blk, "\n")
if len(rows) ≠ 2 * d[1] + 1 {
    return t, fmt.Errorf("block␣%d:␣want␣%d␣rows,␣got␣%d", x, 2 * d[1] + 1, len(rows))
}
for y, row := range rows {
    if len(row) ≠ 2 * d[2] + 1 {
        return t, fmt.Errorf("block␣%d␣row␣%d:␣%d␣columns", x, y, len(row))
    }
    for z := 0; z < len(row); z++ {
        isFace := x % 2 + y % 2 + z % 2 ≡ 2
        if isFace ≡ (row[z] ≡ '.') {
            return t, fmt.Errorf("(%d,%d,%d)␣is␣%q", x, y, z, row[z])
        }
        if isFace {
            t.col[pt{x, y, z}] = row[z]
        }
    }
}
```

This code is used in section 51.

53. ⟨ Functions 7 ⟩ +≡

```
func checkPrinted(b brick, blocks []string) {
    t, err := readPrinted(b, blocks)
    if err ≠ Λ {
        fmt.Printf("␣␣malformed:␣%v\n", err)
        return
    }
    bc := byColour()
    used := map[string]bool{}
    ok := true
    ⟨ Look at the cube in every cell 55 ⟩
    fmt.Printf("␣␣%dx%dx%d:␣%d␣cells,␣%d␣distinct␣cubes",
        b.l, b.m, b.n, b.ncells(), len(used))
    fmt.Printf(",␣sides␣%s,␣parallel␣rule␣%v␣--␣%s\n",
        t.sides(), t.oppositeOnly(), verdict[ok ∧ len(used) ≡ b.ncells()])
}
```

54. ⟨ Declarations 5 ⟩ +≡

```
var verdict = map[bool]string{true: "a␣genuine␣brick", false: "SOMETHING␣IS␣WRONG"}
```

55. ⟨ Look at the cube in every cell 55 ⟩ ≡

```
b.eachCell(func(i, j, k int) {
    ff := b.faces(i, j, k)
    var col [6]byte
    for d := 0; d < 6; d++ {
        col[d] = t.col[ff[d]]
    }
    nm, isCube := bc[col]
    if ¬isCube {
        fmt.Printf("␣␣cell␣(%d,%d,%d)␣holds␣%s,␣which␣is␣no␣cube\n", i, j, k, string(col[:]))
        ok = false
        return
    }
    if used[nm] {
        fmt.Printf("␣␣cube␣%s␣appears␣twice\n", nm)
        ok = false
    }
    used[nm] = true
})
```

This code is used in section 53.

56. Handedness. The answer names the cubes in five places: the four cells of the $1 \times 2 \times 2$ array, and one cell of the magnificent brick, whose option it writes out as `135 acbefd 035:a 125:b 134:d 136:f 145:e 235:c`. Those names are chiral—a cube and its mirror image get different ones—while an array of coordinates is just an array until somebody says how it sits in space. Nothing in the exercise or the answer says.

So I read the five cells both ways round. One handedness gives the five names the answer prints and the other gives their five mirror images, which settles which reading was meant even though it was never stated. Reflecting a single axis is enough to change hands, and since the name already ignores rotations it does not matter which axis.

⟨ Functions 7 ⟩ +≡

```
func mirrorName(bc map[[6]byte]string, col [6]byte) string {
    col[dLeft], col[dRight] = col[dRight], col[dLeft]
    return bc[col]
}
```

57. ⟨ Functions 7 ⟩ +≡

```
func printedCheck() {
    fmt.Println("the␣array␣of␣the␣1x2x2␣brick:")
    checkPrinted(brick{1, 2, 2}, printed122)
    fmt.Println("the␣magnificent␣2x3x5␣brick:")
    checkPrinted(brick{2, 3, 5}, printed235)
    fmt.Println("the␣cells␣the␣answer␣names,␣read␣both␣ways␣round:")
    ⟨ Name the five cells the answer names 59 ⟩
}
```

58. ⟨ Declarations 5 ⟩ +≡

```
var namedCells = []struct {
    b brick
    i int
    j int
    k int
    want string
}{
    {brick{1, 2, 2}, 0, 0, 0, "abcedf"},
    {brick{1, 2, 2}, 0, 0, 1, "abcefd"},
    {brick{1, 2, 2}, 0, 1, 0, "abcdfe"},
    {brick{1, 2, 2}, 0, 1, 1, "abcdef"},
    {brick{2, 3, 5}, 0, 1, 2, "acbefd"},
}
```

59. ⟨ Name the five cells the answer names 59 ⟩ ≡

```
bc := byColour()
right, left := 0, 0
for _, c := range namedCells {
    blocks := printed122
    if c.b.l ≡ 2 {
        blocks = printed235
    }
    t, err := readPrinted(c.b, blocks)
    if err ≠ Λ {
        fmt.Println(err)
        continue
    }
    ff := c.b.faces(c.i, c.j, c.k)
    var col [6]byte
    for d := 0; d < 6; d++ {
        col[d] = t.col[ff[d]]
    }
    ⟨ Compare the two readings with the printed name 60 ⟩
}
fmt.Printf("␣␣right-handed␣agrees␣%d␣times␣out␣of␣%d,␣left-handed␣%d\n",
    right, len(namedCells), left)
```

This code is used in section 57.

60. ⟨ Compare the two readings with the printed name 60 ⟩ ≡

```
r, l := bc[col], mirrorName(bc, col)
if r ≡ c.want {
    right++
}
if l ≡ c.want {
    left++
}
fmt.Printf("␣␣(%d,%d,%d)␣right-handed␣%s,␣left-handed␣%s,␣the␣answer␣says␣%s\n",
    2 * c.i + 1, 2 * c.j + 1, 2 * c.k + 1, r, l, c.want)
```

This code is used in section 59.

61. Two near misses. The answer ends by pointing out that although no $3 \times 3 \times 3$ brick exists, one can come close: the shape with a corner missing can be built from 26 of the cubes, and the shape without the middle cube and the one above it from 25. Leaving cells out is all this takes; the faces where a cell is missing simply belong to nothing, and the sides that remain are still required to be solid.

```
⟨ Functions 7 ⟩ +≡
    func nearMiss( ) {
        for _, c := range nearShapes {
            skip = map[[3]int]bool{ }
            for _, g := range c.gone {
                skip[g] = true
            }
            b := brick{3, 3, 3}
            ts := b.solve( )
            ⟨ Say how the near miss came out 63 ⟩
        }
        skip = map[[3]int]bool{ }
    }
```

62.

```
⟨ Declarations 5 ⟩ +≡
    var nearShapes = []struct {
        what string
        gone [][3]int
    }{
        {"a 3x3x3 with one corner missing", [][3]int{{2, 2, 2}}},
        {"a 3x3x3 without the middle cube and the one above it",
            [][3]int{{1, 1, 1}, {1, 1, 2}}},
    }
```

63.

```
⟨ Say how the near miss came out 63 ⟩ ≡
    fmt.Printf("%s: %d cells, %d solutions", c.what, b.ncells( ), len(ts))
    if len(ts) > 0 {
        bc := byColour( )
        used := map[string]bool{ }
        b.eachCell(func(i, j, k int) {
            ff := b.faces(i, j, k)
            var col [6]byte
            for d := 0; d < 6; d++ {
                col[d] = ts[0].col[ff[d]]
            }
            used[bc[col]] = true
        })
        fmt.Printf(", the first built from %d of the thirty cubes", len(used))
    }
    fmt.Println( )
```

This code is used in section 61.

64. A count that shares nothing. Section 6 of the notes says that a second program, written separately, agreed on the 22. That program never made it into this directory, so in November 2026 I wrote another one here, after a bug in my reading of exercise 334 had hidden behind checks that all shared one step. It shares nothing with the rest of this file: not the naming rule, not the coordinates of exercise 145, not the exact cover engine, not the classification. It knows only that a cube is a way of painting six colours on six faces, counted up to rotation, and it builds bricks by plain backtracking, one cell at a time.

The faces of a cell are numbered $+x$, $-x$, $+y$, $-y$, $+z$, $-z$. A rotation is a permutation of those six, and two quarter turns generate all twenty-four. Turning a painting by a rotation moves the colour of face f to face $r[f]$.

```
⟨ Functions 7 ⟩ +≡
    func bruteSides(l, m, n int) {
        ⟨ Make the twenty-four rotations, from two quarter turns 65 ⟩
        ⟨ Sort the 720 paintings into thirty cubes 66 ⟩
        ⟨ Index the paintings by their colours on the low faces 67 ⟩
        ⟨ Build the bricks cell by cell, with the first cell pinned 68 ⟩
        fmt.Printf("%dx%dx%d by plain backtracking: %d bricks;" +
            " by number of outside colours %v\n", l, m, n, total, byCount)
    }
```

65. ⟨ Make the twenty-four rotations, from two quarter turns 65 ⟩ ≡

```
    qx := [6]int{0, 1, 4, 5, 3, 2}
    qz := [6]int{2, 3, 1, 0, 4, 5}
    turnsOf := map[[6]int]bool{{0, 1, 2, 3, 4, 5}: true}
    for grown := true; grown; {
        grown = false
        for r := range turnsOf {
            for _, q := range [][6]int { qx, qz }{
                var t [6]int
                for f := range 6 {
                    t[f] = q[r[f]]
                }
                if ¬turnsOf[t] {
                    turnsOf[t], grown = true, true
                }
            }
        }
    }
```

This code is used in section 64.

66. A painting is a string of six colours, one per face. Every painting of a cube is one of its twenty-four turnings, so going through all 720 and marking each turning of every new one sorts them into cubes; there should be thirty.

⟨ Sort the 720 paintings into thirty cubes 66 ⟩ ≡

```
var paints []string
var perm func(pre string, rest string)
perm = func(pre, rest string) {
    if rest ≡ "" {
        paints = append(paints, pre)
    }
    for i := range len(rest) {
        perm(pre + rest[i:i + 1], rest[:i] + rest[i + 1:])
    }
}
perm("", "abcdef")
cubeOf := map[string]int{}
nc := 0
for _, p := range paints {
    if _, ok := cubeOf[p]; ok {
        continue
    }
    for r := range turnsOf {
        t := []byte(p)
        for f := range 6 {
            t[r[f]] = p[f]
        }
        cubeOf[string(t)] = nc
    }
    nc++
}
if nc ≠ 30 ∨ len(cubeOf) ≠ 720 {
    panic("the␣paintings␣do␣not␣make␣thirty␣cubes")
}
```

This code is used in section 64.

67. A cell's three low faces must match whatever lies below, behind and to the left of it, so the paintings are filed by those three colours, with a dot where there is no neighbour yet.

⟨ Index the paintings by their colours on the low faces 67 ⟩ ≡

```
lowKey := func(p string, has [3]bool) string {
    k := []byte("...")
    for i, f := range []int { 1, 3, 5 }{
        if has[i] {
            k[i] = p[f]
        }
    }
    return string(k)
}
filed := map[[3]bool]map[string][]string{ }
for _, has := range [][3]bool {{false, false, false}, {false, false, true},
    {false, true, false}, {false, true, true}, {true, false, false},
    {true, false, true}, {true, true, false}, {true, true, true} }{
        filed[has] = map[string][]string{ }
        for _, p := range paints {
            filed[has][lowKey(p, has)] = append(filed[has][lowKey(p, has)], p)
        }
    }
```

This code is used in section 64.

68. The six outer sides must each be one colour, so the colour of a side is fixed by the first cell that touches it, and every later cell must agree.

⟨ Build the bricks cell by cell, with the first cell pinned 68 ⟩ ≡

```
var cs [][3]int
for i := range l {
    for j := range m {
        for k := range n {
            cs = append(cs, [3]int{i, j, k})
        }
    }
}
at := map[[3]int]string{ }
side := map[int]byte{ }
used := map[int]bool{ }
total, byCount := 0, map[int]int{ }
var place func(c int)
place = func(c int) {
    if c == len(cs) {
        ⟨ Tally one brick by its outside colours 69 ⟩
        return
    }
    ⟨ Try every painting that fits cell cs[c] 70 ⟩
}
place(0)
```

This code is used in section 64.

69. ⟨ Tally one brick by its outside colours 69 ⟩ ≡

```
seen := map[byte]bool{ }
for _, x := range side {
    seen[x] = true
}
total++
byCount[len(seen)]++
```

This code is used in section 68.

70. ⟨ Try every painting that fits cell cs[c] 70 ⟩ ≡

```
i, j, k := cs[c][0], cs[c][1], cs[c][2]
var has [3]bool
want := []byte("...")
for x, nb := range [][3]int {{i − 1, j, k}, {i, j − 1, k}, {i, j, k − 1} }{
    if p, ok := at[nb]; ok {
        has[x], want[x] = true, p[[]int{0, 2, 4}[x]]
    }
}
outer := []int{ }
for _, s := range [][2]int {{1, i}, {0, l − 1 − i}, {3, j}, {2, m − 1 − j},
    {5, k}, {4, n − 1 − k} }{
        if s[1] ≡ 0 {
            outer = append(outer, s[0])
        }
    }
cand := filed[has][string(want)]
if c ≡ 0 {
    cand = []string{"abcdef"}
}
for _, p := range cand {
    ⟨ Place p in cell cs[c] if its cube is free and its outer sides agree 71 ⟩
}
```

This code is used in section 68.

71. ⟨ Place p in cell $cs[c]$ if its cube is free and its outer sides agree 71 ⟩ ≡

```
q := cubeOf[p]
if used[q] {
    continue
}
fits := true
var fresh []int
for _, f := range outer {
    if x, ok := side[f]; ¬ok {
        fresh = append(fresh, f)
    } else if x ≠ p[f] {
        fits = false
    }
}
if ¬fits {
    continue
}
for _, f := range fresh {
    side[f] = p[f]
}
used[q], at[cs[c]] = true, p
place(c + 1)
delete(at, cs[c])
used[q] = false
for _, f := range fresh {
    delete(side, f)
}
```

This code is used in section 70.

72. Index.

a: 11, 33, 36, 38, 39, 45.
accounted: 42.
append: 8, 9, 11, 15, 21, 23, 25, 30, 33, 43, 45, 47, 66, 67, 68, 70, 71.
at: 68, 70, 71.
aut: 42, 43.
axisPerms: 10, 12, 32, 37.
b: 19, 20, 21, 22, 23, 24, 25, 27, 28, 29, 30, 32, 34, 35, 36, 37, 40, 42, 46, 51, 53, 55, 58, 59, 61, 63.
bad: 41.
bc: 16, 53, 55, 56, 59, 60, 63.
best: 35, 37, 38.
blk: 51, 52.
blocks: 51, 53, 59.
Bool: 2.
brick: 3, 4, 18, 19, 20, 21, 22, 24, 29, 30, 32, 34, 40, 46, 51, 53, 57, 58, 61.
bruteSides: 3, 64.
Builder: 24, 34, 38.
byColour: 15, 16, 53, 59, 63.
byCount: 64, 68, 69.
c: 8, 9, 15, 16, 25, 28, 34, 35, 38, 39, 42, 47, 59, 60, 61, 63, 68, 70, 71.
cand: 70.
canon: 35, 42, 45.
cells: 1, 30, 46.
census: 3, 16.
checkPrinted: 53, 57.
class: 42, 45.
cnt: 39.
col: 6, 8, 14, 15, 27, 29, 31, 34, 35, 36, 52, 55, 56, 59, 60, 63.
contains: 43, 44.
coords: 21, 34, 36.
cs: 16, 68, 70, 71.
cube: 6, 7, 8.
cubeOf: 66, 71.
cubes: 7, 15, 16, 25.
d: 11, 15, 21, 27, 28, 32, 33, 36, 37, 47, 51, 52, 55, 59, 63.
Dance: 30, 46.
dBack: 6, 23.
dBot: 6, 23.
delete: 71.
det: 11.
dFront: 6, 23.
dims: 19, 21, 32, 36, 37, 51.
dirOf: 11, 13.
distinctSides: 3, 46.
dLeft: 6, 23, 56.
dRight: 6, 23, 56.
dTop: 6, 23.
dump: 2.
dumpAll: 2, 5, 43.
dvec: 6, 11, 13.
eachCell: 19, 22, 25, 27, 55, 63.
err: 53, 59.
Errorf: 51, 52.
ext: 28.
exterior: 22, 28.
f: 19, 25, 28, 32, 33, 64, 65, 66, 67, 71.
faceOn: 36.
faces: 20, 22, 25, 27, 55, 59, 63.
ff: 22, 23, 27, 55, 59, 63.
filed: 67, 70.
first: 27.
fits: 71.
fl: 33.
flag: 2.
flip: 8.
fmt: 2, 3, 16, 20, 25, 27, 28, 31, 40, 41, 42, 43, 45, 46, 47, 51, 52, 53, 55, 57, 59, 60, 63, 64.
fname: 20, 25, 27, 28.
four: 8.
Fprintf: 27, 28.
fresh: 71.
fs: 25.
g: 35, 61.
gone: 61, 62.
grown: 65.
has: 67, 70.
HasPrefix: 47.
head: 47.
i: 4, 13, 19, 20, 22, 23, 25, 27, 31, 47, 55, 58, 59, 60, 63, 66, 67, 68, 70.
img: 35.
in: 30, 46.
IndexByte: 31.
isCube: 55.
isFace: 52.
it: 31.
j: 4, 13, 19, 20, 22, 23, 25, 27, 55, 58, 59, 60, 63, 68, 70.
Join: 24, 46, 47.

k: 4, 19, 20, 22, 23, 25, 27, 39, 43, 45, 55, 58, 59, 60, 63, 67, 68, 70.
key: 42, 43.
keys: 45.
l: 2, 3, 18, 19, 23, 40, 46, 53, 59, 60, 64, 68, 70.
last: 8.
left: 59, 60.
len: 16, 34, 38, 40, 42, 45, 47, 51, 52, 53, 59, 63, 66, 68, 69.
lim: 36.
lines: 46, 47.
ln: 47.
lowKey: 67.
m: 2, 3, 18, 19, 23, 40, 46, 53, 64, 68, 70.
main: 1.
me: 35.
mirrorName: 56, 60.
missing: 16.
mode: 2, 3.
n: 2, 3, 18, 19, 23, 40, 46, 53, 64, 68, 70.
na: 42, 45.
name: 6, 8, 14, 15, 16, 25, 27.
named: 16.
namedCells: 58, 59.
naut: 35.
nb: 70.
nc: 66.
ncells: 19, 40, 53, 63.
nearMiss: 3, 61.
nearShapes: 61, 62.
NewReader: 30, 46.
NewXCC: 30, 46.
nm: 55.
noPin: 2, 5, 27.
nopin: 2.
nopt: 24, 27, 28, 40.
o: 30, 31.
ok: 34, 38, 53, 55, 66, 70, 71.
oppositeOnly: 39, 41, 53.
others: 8, 9.
out: 7, 8, 9, 10, 11, 15, 21, 22, 23, 30, 32, 33.
outer: 70, 71.
p: 9, 10, 11, 13, 15, 20, 27, 31, 33, 34, 35, 36, 66, 67, 70, 71.
paints: 66, 67.
panic: 13, 66.
Parse: 2.
pats: 43.
perm: 66.
permSign: 11, 13.
pinned: 27.
pl: 16, 27.
place: 68, 71.
placement: 14, 15.
placements: 15, 16, 27.
pr: 36, 37, 38, 39, 45.
pre: 66.
prim: 24, 25.
printedCheck: 3, 57.
printedNames: 16, 17.
printed122: 48, 57, 59.
printed235: 48, 57, 59.
Printf: 16, 40, 41, 42, 43, 45, 46, 53, 55, 59, 60, 63, 64.
Println: 3, 43, 57, 59, 63.
pt: 18, 20, 21, 22, 29, 30, 31, 32, 33, 34, 35, 51, 52.
q: 9, 11, 33, 65, 71.
qx: 65.
qz: 65.
r: 9, 15, 60, 64, 65, 66.
readPrinted: 51, 53, 59.
relabel: 34, 35.
report: 3, 4, 40.
rest: 66.
right: 59, 60.
rotations: 10, 15.
row: 52.
rows: 52.
s: 9, 10, 11, 13, 32, 33, 35, 37, 38, 43, 44, 70.
sb: 24, 27, 28, 34, 38.
sec: 24, 25.
seen: 25, 34, 38, 69.
set: 45.
sg: 33.
side: 68, 69, 71.
sideName: 25, 26, 28, 47.
sidePairs: 36, 37, 39, 45.
sides: 37, 42, 43, 45, 53.
size: 2.
skip: 18, 19, 61.
sol: 30.
Solutions: 30, 46.
solve: 30, 40, 61.
somePair: 39.
sort: 25, 43, 45.
Split: 46, 52.

SplitN: 47.
Sprintf: 20, 25, 47.
ss: 44.
Sscanf: 2, 31.
String: 2, 24, 34, 38.
Strings: 25, 43, 45.
strings: 24, 30, 31, 34, 38, 46, 47, 52.
syms: 32, 35, 42.
t: 30, 31, 35, 36, 37, 39, 41, 42, 44, 45, 51, 52, 53, 55, 59, 65, 66.
tally: 42, 43.
tensor: 29, 30, 35, 36, 37, 39, 42, 51.
total: 64, 68, 69.
TrimRight: 46.
ts: 40, 41, 42, 61, 63.
turnsOf: 65, 66.
up: 8.
used: 47, 53, 55, 63, 68, 71.
v: 13, 33, 43.
verdict: 53, 54.
vp: 8.
w: 8, 11, 13, 16.
want: 58, 60, 70.
what: 62, 63.
wp: 8.
WriteByte: 27, 28, 34, 38.
WriteString: 24, 28, 38.
x: 21, 51, 52, 69, 70, 71.
xc: 30, 46.
xcc: 24, 30, 40, 46.
y: 21, 52.
z: 21, 52.

⟨ Add a colour item to the header and to every side option 47 ⟩ Used in section 46
⟨ Add the symmetry that permutes by s and reverses by f 33 ⟩ Used in section 32
⟨ Assemble one signed permutation and keep the proper ones 11 ⟩ Used in section 10
⟨ Build the bricks cell by cell, with the first cell pinned 68 ⟩ Used in section 64
⟨ Check the remark about parallel sides 41 ⟩ Used in section 40
⟨ Collect the faces of this cell that face outwards 23 ⟩ Used in section 22
⟨ Compare the two readings with the printed name 60 ⟩ Used in section 59
⟨ Copy this option's face colours into the array 31 ⟩ Used in section 30
⟨ Declarations 5 ⟩ Used in section 1
⟨ Do what the mode asks 3 ⟩ Used in section 1
⟨ Functions 7 ⟩ Used in section 1
⟨ Index the paintings by their colours on the low faces 67 ⟩ Used in section 64
⟨ List every class with its six side colours 45 ⟩ Used in section 43
⟨ Look at the cube in every cell 55 ⟩ Used in section 53
⟨ Make the twenty-four rotations, from two quarter turns 65 ⟩ Used in section 64
⟨ Name the five cells the answer names 59 ⟩ Used in section 57
⟨ Name the items 25 ⟩ Used in section 24
⟨ Place p in cell $cs[c]$ if its cube is free and its outer sides agree 71 ⟩ Used in section 70
⟨ Print one row per pattern of face colours 43 ⟩ Used in section 42
⟨ Read one block of the array 52 ⟩ Used in section 51
⟨ Read the command line 2 ⟩ Used in section 1
⟨ Run every shape that could possibly work 4 ⟩ Used in section 3
⟨ Say how the near miss came out 63 ⟩ Used in section 61
⟨ Sort the 720 paintings into thirty cubes 66 ⟩ Used in section 64
⟨ Sort the solutions into classes and print the line 42 ⟩ Used in section 40
⟨ Spell the three pairs out in the order s 38 ⟩ Used in section 37
⟨ Take each colour in turn as the one opposite a 8 ⟩ Used in section 7
⟨ Tally one brick by its outside colours 69 ⟩ Used in section 68
⟨ The array of the $1 \times 2 \times 2$ brick 49 ⟩ Used in section 48
⟨ The array of the magnificent brick 50 ⟩ Used in section 48
⟨ Try every painting that fits cell $cs[c]$ 70 ⟩ Used in section 68
⟨ Write an option for every cube in every cell 27 ⟩ Used in section 24
⟨ Write the options that paint the outer sides 28 ⟩ Used in section 24

MacMahon Bricks

	Section	Page
Introduction	1	1
The thirty cubes	6	3
Turning a cube over	10	5
Every colouring is a cube	14	7
Coordinates for a brick	18	9
The exact cover problem	24	11
Reading a solution back	29	13
Telling bricks apart	32	14
The face colours	36	16
One line of the catalogue	40	18
Six different colours	46	21
The arrays the answer prints	48	22
Handedness	56	25
Two near misses	61	27
A count that shares nothing	64	28
Index	72	33